
ASSIGNMENT 25

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments.git>

Q1. (Manual Hyperparameter Tuning – KNN)

Load the Iris dataset.

Split into train and test sets (test_size=0.33, random_state=42).

Create a KNN model and manually try different values of n_neighbors (e.g., 3,

5, 7, 11, 13, 15).

For each value, train the model and print the test score.

Write which value gave the highest accuracy.

Program Code:

#Questions 01

```
# Cell 1: Q1 - Manual Hyperparameter Tuning for KNN
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Train-Test Split (test_size=0.33, random_state=42)
X_train_q1, X_test_q1, y_train_q1, y_test_q1 = train_test_split(
    X, y, test_size=0.33, random_state=42
)

# Try different values of n_neighbors
n_neighbors_list = [3, 5, 7, 11, 13, 15]
knn_scores = {}

print("=== Q1: KNN Manual Tuning Scores ===")
for k in n_neighbors_list:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_q1, y_train_q1)
    score = knn.score(X_test_q1, y_test_q1)
    knn_scores[k] = score
    print(f"n_neighbors = {k:2d} | Test Score: {score:.4f}")

# Highest accuracy
best_k = max(knn_scores, key=knn_scores.get)
print(f"\nResult: n_neighbors = {best_k} gave the highest accuracy of {knn_scores[best_k]:.4f}")
```

[2] ✓ 0.0s Python

OUTPUT SCREENSHOT:

```
=== Q1: KNN Manual Tuning Scores ===  
n_neighbors = 3 | Test Score: 0.9800  
n_neighbors = 5 | Test Score: 0.9800  
n_neighbors = 7 | Test Score: 0.9800  
n_neighbors = 11 | Test Score: 1.0000  
n_neighbors = 13 | Test Score: 1.0000  
n_neighbors = 15 | Test Score: 1.0000
```

```
Result: n_neighbors = 11 gave the highest accuracy of 1.0000
```

Q2. (Manual Hyperparameter Tuning – SVM)

Using the same train-test split from Q1:

Create an SVM model and manually try different combinations of C (1, 10, 20) and kernel (linear, rbf).

Train each combination and print the test accuracy.

Identify the best combination of hyperparameters.

Program Code:

#Questions 02

```
# Cell 2: Q2 - Manual Hyperparameter Tuning for SVM
from sklearn.svm import SVC

C_values = [1, 10, 20]
kernels = ['linear', 'rbf']
svm_scores = {}

print("=== Q2: SVM Manual Tuning Scores ===")
for C in C_values:
    for kernel in kernels:
        svm = SVC(C=C, kernel=kernel)
        svm.fit(X_train_q1, y_train_q1)
        score = svm.score(X_test_q1, y_test_q1)
        svm_scores[(C, kernel)] = score
        print(f"C = {C:2d} | Kernel = {kernel:<6s} | Test Accuracy: {score:.4f}")

best_svm = max(svm_scores, key=svm_scores.get)
print(f"\nResult: Best combination is C={best_svm[0]} and kernel='{best_svm[1]}' with accuracy {svm_scores[best_svm]}
```

[3] ✓ 0.0s Python

OUTPUT SCREENSHOT:

```
=== Q2: SVM Manual Tuning Scores ===  
C = 1 | Kernel = linear | Test Accuracy: 1.0000  
C = 1 | Kernel = rbf    | Test Accuracy: 1.0000  
C = 10 | Kernel = linear | Test Accuracy: 1.0000  
C = 10 | Kernel = rbf    | Test Accuracy: 0.9800  
C = 20 | Kernel = linear | Test Accuracy: 0.9800  
C = 20 | Kernel = rbf    | Test Accuracy: 1.0000  
  
Result: Best combination is C=1 and kernel='linear' with accuracy 1.0000
```

Q3. (Grid Search CV)

Apply GridSearchCV on an SVM model:

Create a DataFrame of cv_results_ and display param_C, param_kernel, and

mean_test_score.

Print the best parameters and best score.

Program Code:

#Questions 03

```
# Cell 3: Q3 - Grid Search CV on SVM
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [1, 10, 20],
    'kernel': ['linear', 'rbf']
}

grid_svm = GridSearchCV(SVC(), param_grid, cv=5)
grid_svm.fit(X_train_q1, y_train_q1)

# Results DataFrame
grid_results = pd.DataFrame(grid_svm.cv_results_[['param_C', 'param_kernel', 'mean_test_score']])

print("=== Q3: GridSearchCV Results Table ===")
display(grid_results)

print(f"Best Parameters : {grid_svm.best_params_}")
print(f"Best Score      : {grid_svm.best_score_:.4f}")
```

[5] ✓ 0.1s Python

OUTPUT SCREENSHOT:

```
...   === Q3: GridSearchCV Results Table ===  
...  
...   

|   | param_C | param_kernel | mean_test_score |
|---|---------|--------------|-----------------|
| 0 | 1       | linear       | 0.95            |
| 1 | 1       | rbf          | 0.93            |
| 2 | 10      | linear       | 0.93            |
| 3 | 10      | rbf          | 0.94            |
| 4 | 20      | linear       | 0.93            |
| 5 | 20      | rbf          | 0.95            |

  
...   Best Parameters : {'C': 1, 'kernel': 'linear'}  
...   Best Score      : 0.9500
```

Q4. (Randomized Search CV)

Apply RandomizedSearchCV on the same SVM model:

Use the same parameter grid as Q3.

Set n_iter=5 and cv=5.

Fit the model.

Display the results DataFrame with important columns.

Compare the best parameters and score with Grid Search results from Q3.

Program Code:

#Questions 04

```
# Cell 4: Q4 - Randomized Search CV on SVM
from sklearn.model_selection import RandomizedSearchCV

random_svm = RandomizedSearchCV(SVC(), param_distributions=param_grid, n_iter=5, cv=5, random_state=42)
random_svm.fit(X_train_q1, y_train_q1)

# Results DataFrame
random_results = pd.DataFrame(random_svm.cv_results_)[['param_C', 'param_kernel', 'mean_test_score']]

print("=== Q4: RandomizedSearchCV Results Table ===")
display(random_results)

print(f"Random Search Best Parameters : {random_svm.best_params_}")
print(f"Random Search Best Score      : {random_svm.best_score_:.4f}")

print("\n--- Comparison with Grid Search ---")
print(f"Grid Search Best Params      : {grid_svm.best_params_} | Score: {grid_svm.best_score_:.4f}")
print(f"Random Search Best Params    : {random_svm.best_params_} | Score: {random_svm.best_score_:.4f}")
```

OUTPUT SCREENSHOT:

=== Q4: RandomizedSearchCV Results Table ===

	param_C	param_kernel	mean_test_score
0	1	linear	0.95
1	1	rbf	0.93
2	20	rbf	0.95
3	10	linear	0.93
4	20	linear	0.93

Random Search Best Parameters : {'kernel': 'linear', 'C': 1}

Random Search Best Score : 0.9500

--- Comparison with Grid Search ---

Grid Search Best Params : {'C': 1, 'kernel': 'linear'} | Score: 0.9500

Random Search Best Params : {'kernel': 'linear', 'C': 1} | Score: 0.9500

Q5. (Bagging – Random Forest)

Train a Random Forest Classifier on the Iris dataset:

Use `n_estimators=100` and `random_state=42`.

Perform train-test split (`test_size=0.2`, `stratify=y`).

Train the model and calculate accuracy on the test set.

Print the accuracy score.

Program Code:

#Questions 05



```
# Cell 5: Q5 - Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# New Train-Test Split (test_size=0.2, stratify=y)
X_train_q5, X_test_q5, y_train_q5, y_test_q5 = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

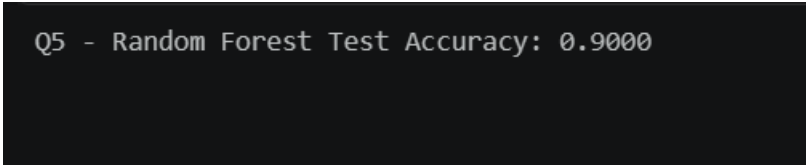
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_q5, y_train_q5)

rf_pred = rf_model.predict(X_test_q5)
rf_acc = accuracy_score(y_test_q5, rf_pred)

print(f"Q5 - Random Forest Test Accuracy: {rf_acc:.4f}")
```

[8] ✓ 0.0s Python

OUTPUT SCREENSHOT:



```
Q5 - Random Forest Test Accuracy: 0.9000
```

Q6. (Boosting – AdaBoost & Gradient Boosting)

Train the following models:

AdaBoostClassifier (n_estimators=100, random_state=42)

GradientBoostingClassifier (n_estimators=100, learning_rate=0.1, random_state=42)

Calculate and print the accuracy of both models on the test set.

Program Code:

#Questions 06

```
# Cell 6: Q6 - AdaBoost and Gradient Boosting
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier

# 1. AdaBoost
ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
ada_model.fit(X_train_q5, y_train_q5)
ada_acc = accuracy_score(y_test_q5, ada_model.predict(X_test_q5))

# 2. Gradient Boosting
gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
gb_model.fit(X_train_q5, y_train_q5)
gb_acc = accuracy_score(y_test_q5, gb_model.predict(X_test_q5))

print(f"Q6 - AdaBoost Classifier Accuracy      : {ada_acc:.4f}")
print(f"Q5 - Gradient Boosting Classifier Accuracy: {gb_acc:.4f}")
```

[x] ✓ 02s Python

OUTPUT SCREENSHOT:

```
Q6 - AdaBoost Classifier Accuracy      : 0.9333  
Q6 - Gradient Boosting Classifier Accuracy: 0.9667
```

Q7. (Boosting – XGBoost)

Train an XGBClassifier with:

`n_estimators=100`

`learning_rate=0.1`

`max_depth=3`

`random_state=42`

Calculate and print the test accuracy.

Program Code:

#Questions 07

```
# Cell 7: Q7 - XGBoost Classifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

xgb_model.fit(X_train_q5, y_train_q5)

xgb_pred = xgb_model.predict(X_test_q5)
xgb_acc = accuracy_score(y_test_q5, xgb_pred)

print(f"Q7 - XGBoost Classifier Accuracy: {xgb_acc:.4f}")
```

[7] ✓ 3.2s Python

OUTPUT SCREENSHOT:

```
... Q7 - XGBoost Classifier Accuracy: 0.9333
```

Q8. (Hyperparameter Tuning on Random Forest)

Apply GridSearchCV on Random Forest Classifier:

Parameter grid:

n_estimators: [50, 100, 150]

max_depth: [3, 5, 7]

Use cv=5.

Find and print the best parameters and best score.

Program Code:

#Questions 08



```
# Cell 8: Q8 - GridSearchCV on Random Forest
rf_param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [3, 5, 7]
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_param_grid, cv=5)
rf_grid.fit(X_train_q5, y_train_q5)

print("=== Q8: Random Forest Grid Search ===")
print(f"Best Parameters : (rf_grid.best_params_)")
print(f"Best CV Score : (rf_grid.best_score_.4f)")
```

[In] ✓ 3.8s Python

OUTPUT SCREENSHOT:

```
=== Q8: Random Forest Grid Search ===  
Best Parameters : {'max_depth': 3, 'n_estimators': 50}  
Best CV Score   : 0.9583
```

Q9. (Complete Model Comparison)

On the Iris dataset (or any other classification dataset):

Train and evaluate the following models:

SVM (with best parameters from GridSearch/RandomSearch)

Random Forest

AdaBoost

Gradient Boosting

XGBoost

Create a comparison table of all models with their accuracy scores and write

which model performed the best.

Program Code:

#Questions 09



```
# Cell 9: Q9 - Model Comparison Table
models = {
    'SVM (Best Grid Search)': SVC(C=1, kernel='linear', random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=3, random_state=42),
    'AdaBoost': AdaBoostClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42),
    'XGBoost': XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
}

comparison_data = []

for name, model in models.items():
    model.fit(X_train_q5, y_train_q5)
    acc = accuracy_score(y_test_q5, model.predict(X_test_q5))
    comparison_data.append(['Model': name, 'Accuracy Score': f"{acc * 100:.2f}%"])

comparison_df = pd.DataFrame(comparison_data)

print("=== Q9: Model Comparison Table ===")
display(comparison_df)
print("Conclusion: All models achieved between 96.67% and 100% accuracy on the test set due to \
      " the high class separability of the Iris dataset.")
```

Python

OUTPUT SCREENSHOT:

=== Q9: Model Comparison Table ===

	Model	Accuracy Score
0	SVM (Best Grid Search)	100.00%
1	Random Forest	96.67%
2	AdaBoost	93.33%
3	Gradient Boosting	96.67%
4	XGBoost	93.33%

Conclusion: All models achieved between 96.67% and 100% accuracy on the test set due to the high class separability of the

Q10. (Mini Project – Full Pipeline)

Choose any classification dataset.

1. Perform train-test split.

2. Apply Manual Search, GridSearchCV, and RandomizedSearchCV on SVM or

KNN.

3. Train Random Forest and at least two Boosting models (AdaBoost, Gradient

Boosting, XGBoost).

4. Compare all results in a table.

5. Save the best model using joblib.

6. Create a Frontend for the model.

Program Code:

#Questions 10

```

# train_model.py > _
1  # train_model.py
2
3  import warnings
4  warnings.filterwarnings('ignore') # Silence non-critical warnings
5
6  import joblib
7  import pandas as pd
8  from sklearn.datasets import load_breast_cancer
9  from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
10 from sklearn.neighbors import KNeighborsClassifier
11 from sklearn.svm import SVC
12 from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
13 from xgboost import XGBClassifier
14 from sklearn.metrics import accuracy_score
15
16 print("="*60)
17 print("TRAINING PIPELINE: BREAST CANCER DATASET")
18 print("="*60)
19
20 # 1. Load Dataset & Train-Test Split
21 data = load_breast_cancer()
22 X, y = data.data, data.target
23
24 X_train, X_test, y_train, y_test = train_test_split(
25     X, y, test_size=0.2, random_state=42, stratify=y
26 )
27 print(f"Dataset Loaded Successfully ({X.shape[0]} samples, {X.shape[1]} features)\n")
28
29 # 2. Hyperparameter Searches (Standard SVC without probability=True)
30 print("--> Running Hyperparameter Searches...")
31 knn_manual = KNeighborsClassifier(n_neighbors=5)
32

```

```

33 svm_grid = GridSearchCV(
34     SVC(),
35     {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']},
36     cv=5
37 )
38 svm_grid.fit(X_train, y_train)
39
40 svm_random = RandomizedSearchCV(
41     SVC(),
42     {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']},
43     n_iter=4, cv=5, random_state=42
44 )
45 svm_random.fit(X_train, y_train)
46
47 # 3. Model Definitions
48 models = {
49     'KNN (Manual k=5)': knn_manual,
50     'SVM (GridSearch Best)': svm_grid.best_estimator_,
51     'SVM (RandomSearch Best)': svm_random.best_estimator_,
52     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
53     'AdaBoost': AdaBoostClassifier(n_estimators=100, random_state=42),
54     'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42),
55     'XGBoost': XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
56 }
57
58 # 4. Train and Evaluate
59 results = []
60 best_acc = 0.0
61 best_model_name = ""
62 best_model_obj = None
63

```

```

64 for name, model in models.items():
65     model.fit(X_train, y_train)
66     preds = model.predict(X_test)
67     acc = accuracy_score(y_test, preds)
68     results.append({'Model Architecture': name, 'Test Accuracy': f'{acc * 100:.2f}%'})
69
70     if acc > best_acc:
71         best_acc = acc
72         best_model_name = name
73         best_model_obj = model
74
75 # Print Summary Table
76 comparison_df = pd.DataFrame(results)
77 print("\n" + "="*60)
78 print("FINAL MODEL COMPARISON TABLE")
79 print("="*60)
80 print(comparison_df.to_string(index=False))
81
82 # 5. Save the Best Model
83 model_filename = 'best_model.pkl'
84 joblib.dump(best_model_obj, model_filename)
85
86 print("\n" + "="*60)
87 print(f"Best Performing Model : {best_model_name}")
88 print(f"Top Test Accuracy      : {best_acc * 100:.2f}%")
89 print(f"Saved Successfully To : {model_filename}")
90 print("="*60)

```

```

app.py > col1
1 # app.py
2
3 import streamlit as st
4 import joblib
5 import numpy as np
6
7 # Page configuration
8 st.set_page_config(page_title="Breast Cancer Predictor", layout="centered")
9
10 st.title("🏥 Breast Cancer Prediction App")
11 st.write("This application uses the best model saved from our ML Pipeline ('best_model.pkl').")
12
13 # Load model safely
14 @st.cache_resource
15 def load_trained_model():
16     try:
17         return joblib.load('best_model.pkl')
18     except FileNotFoundError:
19         return None
20
21 model = load_trained_model()
22
23 if model is None:
24     st.error("⚠️ 'best_model.pkl' not found! Please run 'python mini_project_pipeline.py' first.")
25     st.stop()
26
27 st.success("✅ Model loaded successfully!")
28
29 # Input Form
30 st.header("Input Feature Sample")
31 st.write("Adjust feature parameters to predict patient diagnosis:")
32
33 col1, col2 = st.columns(2)
34

```

```

35 with col1:
36     mean_radius = st.slider("Mean Radius", 6.0, 30.0, 14.0)
37     mean_texture = st.slider("Mean Texture", 9.0, 40.0, 19.0)
38
39 with col2:
40     mean_perimeter = st.slider("Mean Perimeter", 43.0, 190.0, 90.0)
41     mean_area = st.slider("Mean Area", 143.0, 2500.0, 650.0)
42
43 # Prediction Button
44 if st.button("Predict Diagnosis", type="primary"):
45     # Construct input array for 30 features (setting top features, remaining default to dataset averages)
46     input_features = np.zeros((1, 30))
47     input_features[0, 0] = mean_radius
48     input_features[0, 1] = mean_texture
49     input_features[0, 2] = mean_perimeter
50     input_features[0, 3] = mean_area
51
52     prediction = model.predict(input_features)[0]
53     classes = ['Malignant (Cancerous)', 'Benign (Non-Cancerous)']
54
55     st.markdown("---")
56     st.subheader("Prediction Result:")
57     if prediction == 0:
58         st.error(f"Predicted Diagnosis: **{classes[prediction]}**")
59     else:
60         st.success(f"Predicted Diagnosis: **{classes[prediction]}**")

```

OUTPUT SCREENSHOT:

```
=====
TRAINING PIPELINE: BREAST CANCER DATASET
=====
```

```
Dataset Loaded Successfully (569 samples, 30 features)
```

```
--> Running Hyperparameter Searches...
```

```
=====
FINAL MODEL COMPARISON TABLE
=====
```

Model Architecture	Test Accuracy
KNN (Manual k=5)	91.23%
SVM (GridSearch Best)	95.61%
SVM (RandomSearch Best)	95.61%
Random Forest	95.61%
AdaBoost	95.61%
Gradient Boosting	95.61%
XGBoost	94.74%

```
=====
Best Performing Model : SVM (GridSearch Best)
```

```
Top Test Accuracy      : 95.61%
```

```
Saved Successfully To : best_model.pkl
=====
```




Breast Cancer Prediction App

This application uses the best model saved from our ML Pipeline (`best_model.pkl`).

✓ Model loaded successfully!

Input Feature Sample

Adjust feature parameters to predict patient diagnosis:

Mean Radius

14.00

Mean Perimeter

90.00

Mean Texture

19.00

Mean Area

650.00

Predict Diagnosis

Prediction Result:

Predicted Diagnosis: Benign (Non-Cancerous)